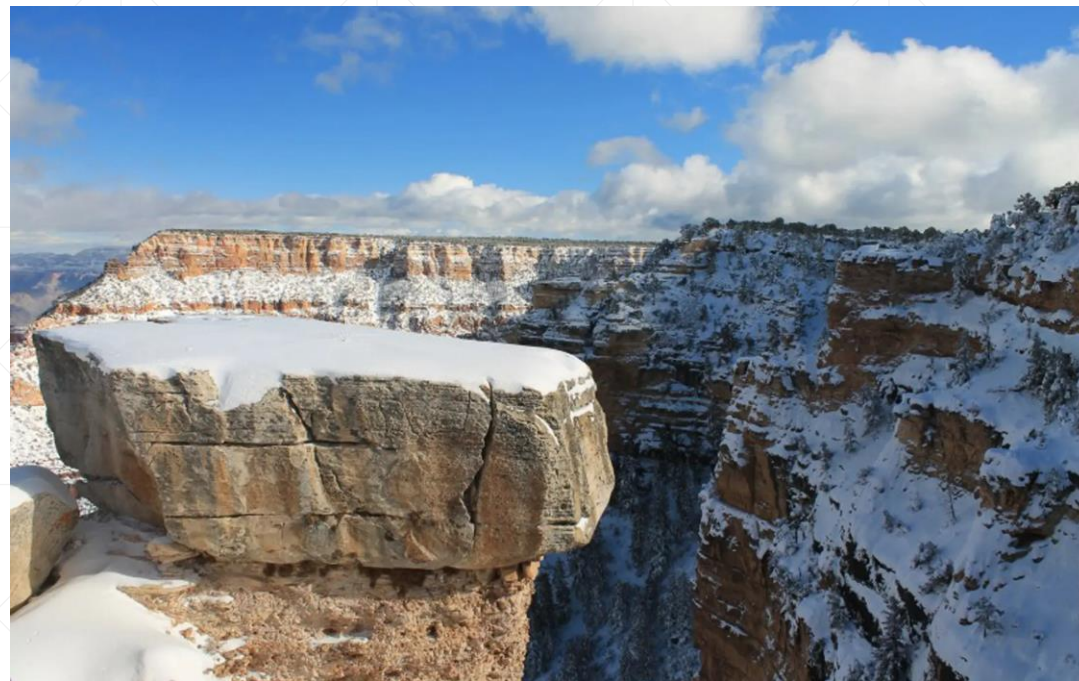




提前中断线程的执行

北京理工大学计算机学院
金旭亮

线程结束（中止）的三种情况



当线程函数执行完毕时，线程自然中止。



当线程中发生未捕获的异常时，操作系统往往会结束整个进程，相应地，线程也会被中止和销毁。



其它线程请求中断当前线程的工作，当前线程响应这个请求而提前结束工作。

向线程发出中断请求



外界通过调用线程的`interrupt()`方法向线程发出中断工作的请求，这时，线程对象内部的一个“中断状态”标记被设置。



线程对象应该**定期检查**这一标记，以合适的方式中断已经运行的工作。

处理外界的中断请求

```
public void run() {  
    try {  
        // 其它代码  
        while (true) {  
            if(!Thread.currentThread().isInterrupted() )  
                // 没有被中断，继续工作.....  
            else  
                break;    // 退出工作  
        }  
        // 其它代码  
    }  
    catch(InterruptedException e) {  
        // 线程在阻塞状态时被中断  
    }  
    finally {  
        // 进行必要的清理工作  
    }  
    // .....  
}
```

外界可通过Thread对象的interrupt()方法，发出“中断请求”。

Thread对象提供了一个isInterrupted()方法，可用于判断外界是否发出了“中断请求”。

线程中断示例：主动Return

```
class TestPrimeGenerator {  
    public static void main(String[] args) {  
        Thread task = new PrimeGenerator();  
        task.start();  
        //等待5秒之后, 发出中断请求  
        try {  
            TimeUnit.SECONDS.sleep( timeout: 5);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        task.interrupt();  
    }  
}
```

此示例的工作线程不断计算一个数是否是质数，输出找到的质数。主线程等待5秒之后发出中断请求。

```
public class PrimeGenerator extends Thread {  
    @Override  
    public void run() {  
        long number = 1L;  
        while (true) {  
            if (isPrime(number)) {  
                System.out.printf("%d 是质数。 \n", number);  
            }  
            // 如果外界发出中断请求  
            if (isInterrupted()) {  
                System.out.println("质数生成过程被打断。");  
                return;  
            }  
            number++;  
        }  
    }  
    //判断一个数是不是质数  
    1 usage  
    private boolean isPrime(long number) {...}  
}
```

PrimeGenerator.java

如果线程正在“休眠”.....



如果线程因为调用`sleep()`、`Object.wait()`等方法处于休眠阻塞状态，而这时其他线程调用它的`interrupt()`方法，那么这个线程会先被唤醒，接着会抛出`InterruptedException`异常。



开发者可以捕获`InterruptedException`异常，然后写代码处理这种中断场景。

休眠时处理外界的中断请求

```
public void run(){
    try{
        ...
        while(中断标记没有被设置) {
            // 干活
            Thread.sleep(dealy);
        }
    }
    catch(InterruptedException e) {
        //线程在休眠期间被中断了
        //这时可以重设中断标记，以保证代码的正确性
        Thread.currentThread().interrupt();
    }
    finally {
        //...
    }
}
```

当处于休眠状态的线程被中断时，会抛出此异常，并且清除掉中断标记。

如果线程函数的工作代码中需要针对中断标记进行特定的处理（比如本例），“中断标记”被清除可能会带来问题，对于这种情况，应该在catch语句块中“重置中断标记”。

线程休眠时中断示例

```
InterruptSleepDemo.java x
1 public class InterruptSleepDemo {
2     public static void main(String[] args) throws InterruptedException {
3         Runnable runnable=()->{
4             while(true){
5                 if(Thread.currentThread().isInterrupted()){
6                     System.out.println("被中断! ");
7                     break;
8                 }
9                 try{
10                     Thread.sleep(2000);
11                 } catch (InterruptedException e) {
12                     System.out.println("休眠被打断");
13                     //重置中断状态
14                     Thread.currentThread().interrupt();
15                 }
16                 Thread.yield();
17             }
18         };
19         Thread thread=new Thread(runnable);
20         thread.start();
21         Thread.sleep(2000);
22         thread.interrupt();
23     }
24 }
```

进入休眠状态

如果注释掉catch块中的interrupt()这句，会发现“被中断”这个字符串没有被输出，并且整个进程没有被结束。

InterruptSleepDemo.java

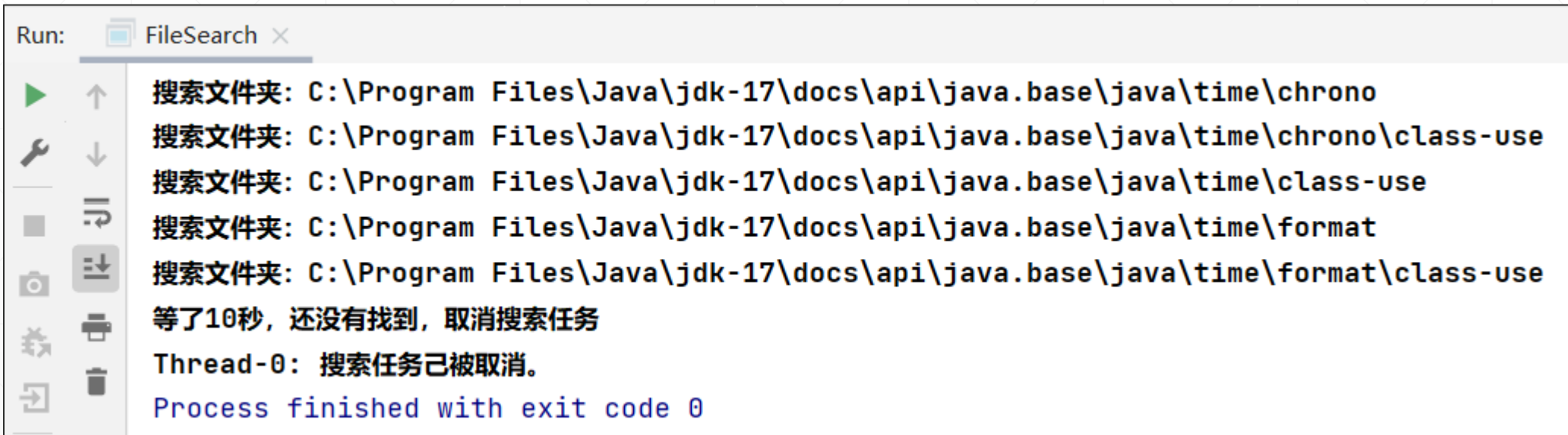
Run: InterruptSleepDemo x

```
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
休眠被打断
被中断!

Process finished with exit code 0
```

应用实例—文件查找

本示例递归检查文件夹中是否包含指定文件，使用Java异常捕获来结束这个搜索过程。



The screenshot shows a 'Run' console window titled 'FileSearch'. It contains the following output:

```
Run: FileSearch x
搜索文件夹: C:\Program Files\Java\jdk-17\docs\api\java.base\java\time\chrono
搜索文件夹: C:\Program Files\Java\jdk-17\docs\api\java.base\java\time\chrono\class-use
搜索文件夹: C:\Program Files\Java\jdk-17\docs\api\java.base\java\time\class-use
搜索文件夹: C:\Program Files\Java\jdk-17\docs\api\java.base\java\time\format
搜索文件夹: C:\Program Files\Java\jdk-17\docs\api\java.base\java\time\format\class-use
等了10秒，还没有找到，取消搜索任务
Thread-0: 搜索任务已被取消。
Process finished with exit code 0
```

支持中断的文件搜索过程-1

directoryProcess函数负责在文件夹中递归搜索文件，注意，在其中插入了“中断请求”检测点。

```
//处理文件
1 usage
private void fileProcess(File file) throws InterruptedException {
    // 比对文件名
    if (file.getName().equals(fileName)) {
        System.out.printf("%s : %s\n",
            Thread.currentThread().getName(),
            file.getAbsolutePath());
    }
    // 如果外界发出了“取消搜索”的请求，则主动抛出InterruptedException
    if (Thread.interrupted()) {
        throw new InterruptedException();
    }
}
```

```
private void directoryProcess(File file)
    throws InterruptedException {
    // 获取此文件夹中的所有文件与子文件夹
    File[] list = file.listFiles();
    if (list != null) {
        for (File value : list) {
            if (value.isDirectory()) {
                System.out.println("搜索文件夹: " + value);
                // 如果是文件夹，则递归搜索它
                directoryProcess(value);
            } else {
                // 如果是文件，看它是不是要找的文件
                fileProcess(value);
            }
        }
    }
    // 如果外界发出了“取消搜索”的请求，则主动抛出InterruptedException
    if (Thread.interrupted()) {
        throw new InterruptedException();
    }
}
```

支持中断的文件搜索过程-2

线程函数:

```
public void run() {  
    File file = new File(initPath);  
    if (file.isDirectory()) {  
        try {  
            //在文件夹中查找文件  
            directoryProcess(file);  
        } catch (InterruptedException e) {  
            System.out.printf("%s: 搜索任务已被取消。",  
                               Thread.currentThread().getName());  
        }  
    }  
}
```

main函数:

```
public static void main(String[] args) {  
    // 在C:盘上使用多线程查找"autoexec.bat"文件  
    FileSearch searcher = new FileSearch( initPath: "C:\\",  
                                           fileName: "autoexec.bat");  
    Thread thread = new Thread(searcher);  
    thread.start();  
    // 等待10秒  
    try {  
        TimeUnit.SECONDS.sleep( timeout: 10);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    System.out.println("等了10秒, 还没有找到, 取消搜索任务");  
    //中断线程  
    thread.interrupt();  
}
```

FileSearch.java

小结：辨析



方法名	说明
<code>void Thread.interrupt()</code>	中断线程
<code>boolean Thread.isInterrupted()</code>	判断是否被中断
<code>static boolean Thread.interrupted()</code>	判断 <code>currentThread()</code> 是否被中断，并清除当前中断状态标记

使用外部标记退出线程



前面介绍的使用`Thread.interrupt`方法退出线程的方法比较复杂，其实，如果线程不会休眠，那么，为线程对象设置一个专门的自定义标记，线程在工作时定时检查一下这个标记，也能达到类似的目的，并且这种方式更为简单直接。



这个标记可以是线程对象的一个私有字段，然后，线程对象必须提供一种手段（通常是提供一个公有方法）来修改它的值。



为了保证线程变量的可见性，这个标记字段应该定义为`volatile`的。

使用外部标记退出线程示例

```
class MyThread extends Thread {  
    2 usages  
    private volatile boolean closeFlag = false;  
    1 usage  
    public void quit(){  
        closeFlag=true;  
    }  
    @Override  
    public void run() {  
        var threadName :String = Thread.currentThread().getName();  
        System.out.println("线程" + threadName + "正在工作");  
        while (!closeFlag) {  
            System.out.print(".");  
            try {...} catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            }  
        }  
        System.out.println("线程" + threadName + "已经退出。");  
    }  
}
```

UseVolatileToBreak.java

```
public class UseVolatileToBreak {  
    public static void main(String[] args) {  
        var thread = new MyThread();  
        thread.start();  
        System.out.println("敲回车键退出...");  
        new Scanner(System.in).nextLine();  
        thread.quit();  
    }  
}
```

Run: UseVolatileToBreak x

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
敲回车键退出...  
线程Thread-0正在工作  
.....  
线程Thread-0已经退出。  
  
Process finished with exit code 0
```

巩固练习



使用多线程在磁盘上查找特定类型的文件（比如图片或音频、视频），最长执行30秒，30秒一到，如果查找任务还没有完成，则中断它！